

AlertLogPkg and ReportPkg User Guide

User Guide for Release 2026.08

By

Jim Lewis

SynthWorks VHDL Training

Jim@SynthWorks.com

<http://www.SynthWorks.com>

Table of Contents

1	AlertLogPkg and ReportPkg Overview	5
2	Simple Mode: Basic Errors and Messaging	6
3	Hierarchy Mode: Tracking Errors and Messages to their sources	8
4	Controlling Log Enables from Other Locations	10
5	Child IDs: Further Classification of IDs	11
6	Multiple Line Printing (2026.08)	12
7	Package References	13
8	Setting the Test Name and AlertLogIDs	14
8.1	SetTestName: Setting the Test Name	14
8.2	GetTestName	14
8.3	GetAlertLogName	14
8.4	AlertLogIDType	14
8.5	AlertLogReportModeType	15
8.6	AlertLogPrintParentType	15
8.7	Creating and Finding AlertLogIDs	15
8.7.1	NewID: Creating a New ID	16
8.7.2	GetID: Find Existing ID or Create it if it does not exist	17
8.7.3	FindID: Find an AlertLogID	17
8.7.4	IsInitialized	17
8.8	PathTail - Used to Discover Instance Name of a Component	17
8.9	GetAlertLogParentID	18
9	Alert Method Reference	19
9.1	AlertType	21
9.2	Alert	21
9.3	AlertIf and AlertIfNot	21
9.4	AlertIfEqual and AlertIfNotEqual	22
9.5	AlertIfFilesNotMatch	22
9.6	IncrementAlertCount	23
10	Log Method Reference	19
10.1	LogType	19
10.2	Log	19
10.3	LogHeader	19
11	Affirmation Reference	24
11.1	AffirmIf / AffirmIfNot	24
11.2	AffirmIfEqual and AffirmIfNotEqual	25
11.3	AffirmIfInRange	25
11.4	AffirmIfFilesMatch	26
11.5	AffirmIfTranscriptsMatch	26

11.6	AffirmIfStable, AffirmIfNotStable.....	27
11.7	GetAffirmCount.....	27
11.8	IncAffirmCount	27
12	HTML Test Reports	28
12.1	Required Steps for Report Generation	28
12.2	EndOfTestReports (in ReportPkg)	28
12.3	SetExpectedAlertCount.....	29
12.4	IncrementExpectedAlertCount	29
12.5	GetExpectedAlertCount	29
13	Requirements Tracking.....	30
13.1	NewReqID.....	30
13.2	GetReqID	31
13.3	ReadSpecification.....	31
13.4	SetPassedGoal	32
13.5	AffirmIf for Requirements.....	32
13.6	Using Other Forms of AffirmIF with Requirements	32
13.7	WriteRequirementsYaml (Called by EndOfTestReports)	33
13.8	Test Failure Due to Missed Requirements	33
13.9	ReportAlerts and Requirements	33
14	Enabling and Disabling Logs and Alerts	35
14.1	SetLogEnable: Enable / Disable Logging.....	35
14.2	Reading Log Enables from a FILE	35
14.3	IsLogEnabled /GetLogEnable	35
14.4	SetAlertEnable: Alert Enable / Disable	36
14.5	GetAlertEnable.....	36
14.6	ClearAlerts: Reset Alert Counts	36
14.7	ClearAlertCounts: Reset Alert and Alert Stop Counts	36
14.8	SetGlobalAlertEnable: Alert Global Enable / Disable.....	36
14.9	GetGlobalAlertEnable	37
15	Stop Count and Print Count	38
15.1	SetAlertStopCount: Alert Stop Counts.....	38
15.2	GetAlertStopCount	38
15.3	ClearAlertStopCounts: Reset Alert Stop Counts	38
15.4	SetAlertPrintCount	39
15.5	GetAlertPrintCount	39
16	Getting Counts.....	40
16.1	AlertCountType.....	40
16.2	GetAlertCount.....	40
16.3	GetEnabledAlertCount	40

16.4	GetDisabledAlertCount	40
16.5	Math on AlertCountType.....	41
16.6	SumAlertCount: AlertCountType to Integer Error Count.....	41
17	Alert and Log Prefix and Suffix to Message	42
18	Text Based Test Case Reports via ReportAlerts	43
18.1	Report Summary.....	43
18.2	Report Details.....	44
18.3	ReportAlerts: Reporting Alerts	44
18.4	ReportNonZeroAlerts.....	45
18.5	Overloaded ReportAlerts for Reporting AlertCounts	45
19	Alert and Log Output Control Options	46
19.1	Settings In OsvvmSettingsPkg_default.vhd	46
19.2	SetAlertLogJustify	46
19.3	OsvvmOptionsType	46
19.4	Deprecated: SetAlertLogOptions	46
19.5	Deprecated: ReportAlertLogOptions: Print Report Options	49
19.6	Deprecated: Getting AlertLog Report Options	49
20	Deallocating and Re-initializing the Data structure.....	49
20.1	DeallocateAlertLogStruct	49
20.2	InitializeAlertLogStruct	50
21	Deprecated: Older CSV Type Reporting	51
21.1	Deprecated Reporting Test Completion: CSV Reports.....	51
21.1.1	Deprecated: WriteAlerts	51
21.1.2	Deprecated: WriteTestSummary	52
21.1.3	Deprecated: ReadTestSummaries	52
21.1.4	Deprecated: ReportTestSummaries	52
21.1.5	Deprecated: WriteTestSummaries.....	53
21.1.6	Deprecated: Use Model	53
21.2	Deprecated: Requirements Reporting: CSV Reports	54
21.2.1	Deprecated: ReportRequirements	54
21.2.2	Deprecated: WriteRequirements	54
21.2.3	Deprecated: ReadRequirements.....	55
22	Deprecated Names and their Replacement name	55
23	About AlertLogPkg	55
24	Compiling and Using OSVVM Utility Library	55
25	About the Author - Jim Lewis.....	56

1 AlertLogPkg and ReportPkg Overview

Error reporting and messaging is an important part of any testbench. AlertLogPkg adds the capabilities of Logs, Alerts, and Affirmations.

Logs are for conditional printing. When debugging, we need messages that detail test activity. However, we do not always want to print the messages as this would slow the simulation down. In particular for regression testing, we want to have only minimal messages – such as pass and fail. As a result, we need to be able to enable and disable messages – and in some cases enable messages from some sources while disabling others.

Alerts are for checking proper behavior or conditions. Use alerts to check that a response (such as ready) is not received when a transfer is not in progress. Alerts are also used to check things like the min parameter is never greater than the max parameter.

Affirmations are for checking that a received (actual) value matches the expected value. If they match, a log PASSED is produced, otherwise, an alert ERROR is produced.

Both Alerts and Affirmations accumulate the number of errors they have detected during a test case. When the test completes these are used to indicate whether the test passed or failed. If the test fails, we need additional information – such as an indication as to which part of the test environment (such as a VC) detected the error.

VHDL assert and TextIO capability can certainly do some of this. Until VHDL-2019, they lack the capability to count errors. Even with VHDL-2019, they lack the capability to indicate which part of the test environment found the error.

OSVVM's Alerts, Logs, and affirmations solve these issues.

2 Simple Mode: Basic Errors and Messaging

Simple mode tracks alerts, logs, and affirmations with a single error and affirmation counters. This is sufficient for basic test cases.

First include OSVVM library in your testbench the OSVVM Context declaration. Context declarations are part of VHDL-2008 and allow a group of packages to be referenced as a single item.

```
Library osvvm ;  
use osvvm.OsvvmContext ;
```

Name the test by calling SetTestName. When using OSVVM scripting, this name must match the test case name provided by the scripting. For simple tests, this is the entity name of the testcase.

```
SetTestName("Tb_SendGet_1") ;
```

Messages in OSVVM are printed using logs.

```
Log("Uart Sending Data: X" & to_hstring(TxData), ALWAYS) ;
```

This will print as:

```
%%          116800.0 ns  Log    ALWAYS    Uart Sending Data: X4A
```

Logs have levels ALWAYS, DEBUG, FINAL, INFO, and PASSED. Levels can be enabled or disabled in the test case. Log level ALWAYS always prints. The remaining logs print when they are enabled and are disabled by default. ALWAYS is the default if a level is not specified.

Enable a log with SetLogEnable.

```
SetLogEnable(INFO, TRUE) ;
```

The following log only prints when the INFO level is enabled and can be turned off for regressions.

```
Log("Uart Sending Data: X" & to_hstring(TxData), INFO) ;
```

This will print as:

```
%%          116800.0 ns  Log    INFO      Uart Sending Data: X4A
```

The same method is used to turn DEBUG or any other message on or off. It is recommended to call SetLogEnable from the top level testbench, however, it can be called from anywhere in the testbench infrastructure.

Alerts are for signaling error conditions. The following is a protocol checker that flags an error if the X86Lite model receives an acknowledge when the interface is idle.

```
X86_Rdy_Check: process  
begin  
    wait until Rising_Edge(Clk) and nRdy = '1' ;  
    AlertIf(BusIdle, "X86 Lite received ready when it was idle", ERROR) ;
```

```
end process ;
```

This will print as:

```
%%          51340.0 ns  Alert  ERROR    X86 Lite received ready when it was idle
```

Alerts have levels FAILURE, ERROR, and WARNING. Error is the default if a level is not specified. Alerts are enabled by default and rarely disabled (see SetAlertEnable).

Affirmations are for checking test values. When the condition is true, the affirmation prints a PASSED message (using log), otherwise, it prints an ERROR message (using alert).

```
AffirmIf(RxData = Expected, "RxData, Actual: " & to_hstring(RxDat),  
        "Expected: " & to_hstring(Expected)) ;
```

Note there are two strings associated with AffirmIf. The first string prints with both PASSED and ERROR. The second string only prints with ERROR messages. When AffirmIf passes and log level "PASSED" is enabled, the following message will print.

```
%%          116800.0 ns  Log    PASSED    RxData, Actual: X4A
```

When AffirmIf is FALSE, the following message will print.

```
%%          116800.0 ns  Alert  ERROR    RxData, Actual: 52 Expected: 4A
```

As Alerts (FAILURE, ERROR, WARNING), PASSED, and affirmations occur, a data structure inside the AlertLogPkg tabulates them.

When the test is done, a text summary message can be created by calling EndOfTestReports. EndOfTestReports also produces a number of YAML reports which OSVVM scripting converts into HTML and JUnit XML reports.

```
EndOfTestReports ;
```

If the test passed, a message such as the following prints. Note the message prints on a single long line that starts with %%%.

```
%%          381100.0 ns  DONE  PASSED    Tb_SendGet_1  Passed: 14  Affirmations  
Checked: 14
```

If the test failed, a message such as the following prints.

```
%%          381100.0 ns  DONE  FAILED    AxiLite_Uart1  Total Error(s) = 6  Failures: 0  
Errors: 6  Warnings: 0  Passed: 8  Affirmations Checked: 14
```

This is enough to get you started with basic testbenches. Read on when you are ready for more capability.

3 Hierarchy Mode: Tracking Errors and Messages to their sources

As testbench complexity increases, it helps to know the source of a message. Often the sources are verification components (VCs). As a result, OSVVM calls this hierarchy mode since the sources reveal the hierarchy of the test.

Just like simple mode, reference OSVVM's context.

```
Library osvvm ;  
use osvvm.OsvvmContext ;
```

Then in the test architecture, set the test name.

```
SetTestName("Tb_SendGet_1") ;
```

In hierarchy mode, an AlertLogID is created for each source. Often these are signals, but variables can also be used.

```
signal ModelID : AlertLogIDType ;
```

IDs are constructed using NewID before usage.

```
Initialize : process  
begin  
    ModelID <= NewID("UartTx_1") ;  
    wait ;  
end process Initialize ;
```

Long term we will be able to do this in one step in a constant. While many simulators currently allow this, the VHDL LRM seems to indicate it is illegal – but it is expected this will be allowed in the next revision of VHDL.

```
constant MODEL_ID : AlertLogIDType := NewID("UartTx_1") ;
```

Ok. The hard part is done. On all calls to Log, SetLogEnable, AlertIf, AffirmIf, SetAlertEnable, ... the AlertLogID is specified as the first parameter to the call.

Log levels can be enabled for specific AlertLogIDs as follows.

```
SetLogEnable(ModelID, INFO, TRUE) ;  
SetLogEnable(TBID, PASSED, TRUE) ;
```

The log in the UartTx VC is updated as follows.

```
Log(ModelID, "Uart Sending Data: X" & to_hstring(TxData), INFO) ;
```

When this prints, note that it includes the source information, "In UartTx,".

```
%%          116800.0 ns  Log  INFO      In UartTx,  Uart Sending Data: X4A
```

The AffirmIf can be replaced by AffirmIfEqual, which internally does the equality check and produces the same message (with less effort).

```
AffirmIfEqual(TBID, RxData, Expected, "RxData, ") ;
```

When AffirmIfEqual passes, log is called to print the following message.


```
%%      116800.0 ns Log PASSED In TB, RxData, Actual: 4A
```

When `AffirmIfEqual` fails, the following message will print.

```
%%      116800.0 ns Alert ERROR In TB, RxData, Actual: 52 Expected: 4A
```

When the test is done, a text summary message can be created by calling `EndOfTestReports`.

```
EndOfTestReports ;
```

If the test passed, we get the same simple message as for Simple Mode.

```
%%      381100.0 ns DONE PASSED Tb_SendGet_1 Passed: 14 Affirmations  
Checked: 14
```

If the test failed, we get a summary as well as details for each source. The line starts with "%%" and prints on a single line (here shown with line wrap to fit in the text).

```
%%      381100.0 ns DONE FAILED AxiLite_Uart1 Total Error(s) = 6 Failures: 0  
Errors: 6 Warnings: 0 Passed: 8 Affirmations Checked: 14  
%%      Default      Failures: 0 Errors: 1 Warnings: 0 Passed: 0  
%%      OSVVM        Failures: 0 Errors: 0 Warnings: 0 Passed: 0  
%%      TB           Failures: 0 Errors: 1 Warnings: 0 Passed: 2  
%%      AxiManager    Failures: 0 Errors: 3 Warnings: 0 Passed: 0  
%%      Protocol      Failures: 0 Errors: 1 Warnings: 0 Passed: 0  
%%      Checker       Failures: 0 Errors: 2 Warnings: 0 Passed: 2  
%%      UartTx        Failures: 0 Errors: 0 Warnings: 0 Passed: 2  
%%      UartRx        Failures: 0 Errors: 1 Warnings: 0 Passed: 2
```

Note errors in the Default bin are the result of calling an alert or affirmation without an `AlertLogID`.

4 Controlling Log Enables from Other Locations

Alerts and Logs may need to be enabled (or disabled) from a test case which is in a different entity than the VC that defined it. To do this, the test case needs to be able to look up the AlertLogID for the VC. No problem. AlertLogIDs can be looked up using NewID and the same string used in its definition. The following code can be added to the test case.

```
signal TBID, AxiManagerID, UartTxID, UartRxID : AlertLogIDType ;
. . .
ControlProc : process
begin
    BID          <= NewID("TB") ;
    wait for 0 ns ;                -- Allow the VC to create the ID
    AxiManagerID <= NewID("AxiManager_1") ; -- These names must match
    UartTxID     <= NewID("UartTx_1") ;    -- the names used in the VC
    UartRxID     <= NewID("UartRx_1") ;
```

Using this capability, we can turn on the INFO messages only for one VC, such as the UartTx VC. Enabling a specific ID is important when the reporting from that VC is infrequent relative to the other VC in the system – such as for a UART which is slow. Without this capability it is a challenge to find UART log messages among messages from faster interfaces.

```
. . .
SetLogEnable(PASSED, TRUE) ;          -- Turn on for ALL Levels
SetLogEnable(UartTxID, INFO, TRUE) ;  -- Turn on just UartTxID
```

5 Child IDs: Further Classification of IDs

Sometimes a VC may wish to further classify its Alerts and Logs into subclasses. This is done as follows. The ID/ModelID is referred to as a parent ID and is constructed without any additional parameters. The ProtocolID and CheckerID are referred to as child IDs (since they belong to the parent).

```
signal ModelID, ProtocolID, CheckerID : AlertLogIDType ;
. . .
Initialize : process
  Variable ID : AlertLogID ;
begin
  ID := NewID("X86Lite") ;
  ModelID <= ID ;
  ProtocolID <= NewID("Protocol", ID) ;
  CheckerID <= NewID("Checker", ID) ;
  wait ;
end process Initialize ;
```

Using ProtocolID, the X86Lite protocol checker is updated as follows.

```
X86_Rdy_Check: process
begin
  wait until Rising_Edge(Clk) and nRdy = '1' ;
  AlertIf(ProtocolID, BusIdle, "X86 Lite received ready when it was idle") ;
end process ;
```

Now the message includes both the parent ID name and child ID name.

```
%%          51340.0 ns  Alert  ERROR  in X86Lite: Protocol,   X86 Lite received ready
when it was
```

Note that when a child ID finds an error, it counts it and its parent also count it.

6 Multiple Line Printing (2026.08)

All OSVVM printing capabilities with Alert, Log, AffirmIf, and LogHeader as well as their variations support multiple line printing.

By default, these print their message on a single line – independent of how long that line may be.

If a message starts with LF or '<', the message will start on the next line with a small indent (ALERT_LOG_INDENTED_LENGTH) and the message will WRAP if the character count exceeds the OSVVM_LINE_WRAP value (currently 165 characters including the %%). The '<' at the start of the line will not be printed. Also note that line wrap was shortened to work inside this document.

```
%%          0.0 ns Log    ALWAYS in Default,
%%>         23456789 323456789 423456789 523456789 623456789 723456789
%%>         3456789 323456789 423456789 523456789 623456789 723456789
%%>         456789 323456789 423456789 523456789 623456789 723456789
```

If a message starts with '>', the message will start after printing the ID. If character count exceeds the OSVVM_LINE_WRAP value, a new line will be added and the next line will start aligned with the location the message started printing. The '>' at the start of the line will not be printed.

```
%%          0.0 ns Log    ALWAYS in Default, 523456789 623456789 723456789
%%>         23456789 623456789 723456789
%%>         3456789 623456789 723456789
%%>         456789 623456789 723456789
```

Otherwise, if the OSVVM setting OSVVM_LOG_WRAP is true, the line will be printed in the same fashion as a line starting with '>'.

Otherwise, if the line contains an LF character, the line will wrap at the LF. The first line of the message is printed immediately following the ID. Additional lines are printed on following lines and are aligned with the start of the message.

Note that when a line is wrapped, the line starts with %%> rather than %%. This allows OSVVM report handlers to know it is a continuation and the HTML red coloring of Alert ERROR can span the entire message rather than just the first line.

As a result of all the checks above, the default printing on a single line happens when the line does not contain a LF, does not start with either '>' or '<', and OSVVM_LOG_WRAP is FALSE.

7 Package References

The easiest way to use OSVVM is to reference its context clause.

```
library osvvm ;  
context osvvm.OsvvmContext ;
```

While individual packages can be referenced, if the OSVVM team refactors packages, your package references will need to be updated.

```
library osvvm ;  
use osvvm.OsvvmGlobalPkg.all ; -- required to call SetAlertLogOptions  
use osvvm.AlertLogPkg.all ;  
use osvvm.ReportPkg.all ;
```

On the same note, calling subprograms with package selected paths (such as `osvvm.ReportPkg.EndOfTestReports`) is not a recommended practice as it will break if the packages are refactored.

8 Setting the Test Name and AlertLogIDs

Tests always start in simple model. Hierarchical mode is automatically entered when NewID is called.

All overloading supported for simple mode is also supported in hierarchical mode. The only difference is that in hierarchical mode, the first parameter is the AlertLogID.

Also note that when in hierarchical mode, any alert, log, or affirmation that is called without an AlertLogID will use the "Default" bin.

8.1 SetTestName: Setting the Test Name

SetTestName sets the name of the current test that is printed by ReportAlerts. Setting the test name is a required part of the OSVVM methodology. It is recommended to set the test name at time 0. If it is not set, the default name is AlertLogTop.

```
procedure SetTestName(Name : string ) ;  
.  
.  
SetTestName("AxiLite_Uart1") ;
```

8.2 GetTestName

GetTestName returns the string value of name associated with a Test.

```
impure function GetTestName return string ;
```

8.3 GetAlertLogName

GetAlertLogName returns the string value of the name associated with an AlertLogID. If no AlertLogID is specified, it will return the name set by SetTestName.

```
impure function GetAlertLogName(AlertLogID : AlertLogIDType:= ALERTLOG_BASE_ID)  
return string ;
```

8.4 AlertLogIDType

AlertLogIDType is defined as a subtype of integer. Using a subtype here works similar to an alias and allows it to be changed as needed.

```
subtype AlertLogIDType is integer ;
```

8.5 AlertLogReportModeType

```
type AlertLogReportModeType is (DISABLED, ENABLED, NONZERO) ;
```

When DISABLED, an AlertLogID does not print in ReportAlerts (text reports) or WriteAlertYaml (Yaml/HTML reports). When NONZERO, an AlertLogID does not print in ReportAlerts, but prints in WriteAlertYaml. Hence, NONZERO shortens the text reports to what is essential. When ENABLED, an AlertLogID prints in ReportAlerts and WriteAlertYaml.

None of these settings impact printing in Alert or Log.

DISABLED is intended to be used for data structures that are not used for self-checking and may produce errors on parameter checks. These errors are not lost as they are still accumulated in the parent ID.

8.6 AlertLogPrintParentType

```
type AlertLogPrintParentType is (PRINT_NAME, PRINT_NAME_AND_PARENT) ;
```

When PRINT_NAME, Alert and Log print the name of the AlertLogID. When PRINT_NAME_AND_PARENT, Alert and Log print the parent ID name, the ID separator, and then the ID name.

8.7 Creating and Finding AlertLogIDs

Each level in a hierarchy is referenced with an AlertLogID. There are five functions used to create or find AlertLogIDs: NewID, GetID, FindID, NewReqID, and GetReqID.

Use NewID when the intent is to create the ID. Use GetID when the intent is to look up the ID – but if it does not exist yet then create it. Use FindID to look up an ID that exists and if it does not, then fail. Use NewReqID when the intent is to create a new requirements ID. Use GetReqID when the intent is to look up the requirements ID – but if it does not exist yet then create it.

The main difference between the NewID and GetID is that NewID uses ALERTLOG_BASE_ID as the default parent ID where as GetID uses ALERTLOG_ID_NOT_ASSIGNED. When the parent ID is ALERTLOG_ID_NOT_ASSIGNED, then the next call to NewID will establish the parent ID. Note though once the parent ID is assigned, it cannot be changed.

NewID is typically called in a verification component. GetID is typically called in the testbench to look up the ID of a verification component. When using GetID it is typical

to call it well after time 0, so as to allow NewID to be called first. If it is indeed well after time 0, then calling FindID will produce the same results as GetID.

8.7.1 NewID: Creating a New ID

Use NewID when the intent is to create the ID, such as in a verification component. If an AlertLogID already exists for the specified name, NewID will return its AlertLogID. It is recommended to use the instance label of a verification component as the Name. When creating an AlertLogID in a verification component, it is recommended to call NewID at time 0.

The interface for NewID is as follows.

```
impure function NewID(  
  Name          : string ;  
  ParentID      : AlertLogIDType      := ALERTLOG_BASE_ID ;  
  ReportMode    : AlertLogReportModeType := ENABLED ;  
  PrintParent   : AlertLogPrintParentType := PRINT_NAME_AND_PARENT ;  
  CreateHierarchy : boolean           := TRUE  
) return AlertLogIDType ;
```

ParentID allows a hierarchy of Alerts to be created with parent and child IDs. Within a verification component it is common to create an ID for the verification component and then create Child IDs that are related to the parent.

```
signal ModelID, ProtocolID, CheckerID : AlertLogIDType ;  
...  
Initialize : process  
  Variable ID : AlertLogID ;  
begin  
  ID := NewID("AxiManager") ;  
  ModelID <= ID ;  
  ProtocolID := NewID("Protocol", ID) ;  
  CheckerID := NewID("Checker", ID) ;  
  wait ;  
end process Initialize ;
```

The ReportMode and PrintParent parameters are described in AlertLogReportModeType and AlertLogPrintParentType.

Setting the CreateHierarchy parameter to FALSE allows packages to create a unique AlertLogID without triggering ReportAlerts to print the hierarchy of the structure – unless the testbench or a verification component also calls NewID. OSVVM data structures set CreateHierarchy to FALSE.

8.7.2 GetID: Find Existing ID or Create it if it does not exist

Use GetID when the intent is to look up the ID – but if it does not exist yet then create it. Typically GetID is called in the testbench, well after time 0, to look up the ID of a verification component.

The interface for GetID is as follows:

```
impure function GetID(  
    Name : string ;  
    ParentID : AlertLogIDType := ALERTLOG_ID_NOT_ASSIGNED ;  
    CreateHierarchy : Boolean := TRUE  
) return AlertLogIDType ;
```

Internally GetID calls NewID:

8.7.3 FindID: Find an AlertLogID

The function, FindID, finds an existing AlertLogID. If the AlertLogID is not found, ALERTLOG_ID_NOT_FOUND is returned. Caution do not use at time 0, as FindID may return ALERTLOG_ID_NOT_FOUND for an ID that has not been initialized by the model yet. Use GetAlertID for this situation.

The interface for FindID is as follows.

```
impure function FindID(Name : string ; ParentID : AlertLogIDType)  
    return AlertLogIDType ;  
impure function FindID(Name : string ) return AlertLogIDType ;
```

FindID with a single parameter will return the first AlertLogID whose name matches the Name parameter, hence, only use this if that name is used once in the AlertLog data structure, such as for top-level instance names.

8.7.4 IsInitialized

IsInitialized returns an indication whether an AlertLogID is initialized or not.

```
impure function IsInitialized (ID : AlertLogIDType) return boolean ;
```

8.8 PathTail - Used to Discover Instance Name of a Component

When used in conjunction with attribute PATH_NAME applied to an entity name, PathTail returns the instance name of component plus any for generate loop indices. For generate loop indices are added as "_#".

```
constant CPU_ALERT_ID : AlertLogIDType :=  
    --          Name (string value),          Parent AlertLogID  
    NewID(PathTail(CpuModel'PATH_NAME), ALERTLOG_BASE_ID) ;
```

8.9 **GetAlertLogParentID**

Get the AlertLogID of the parent of a specified AlertLogID.

```
impure function GetAlertLogParentID(AlertLogID : AlertLogIDType) return  
AlertLogIDType ;
```

9 Log Method Reference

9.1 LogType

Log levels can be ALWAYS, DEBUG, PASSED, or INFO.

```
type LogType is (ALWAYS, DEBUG, FINAL, INFO, PASSED) ;
```

9.2 Log

If the log level is enabled, then the log message will print. The Enable parameter indicates to print the message even if the log level is not enabled. This is useful to enable printing for a single transaction.

```
-- with an AlertLogID
procedure log(
  AlertLogID  : AlertLogIDType ;
  Message     : string ;
  Level       : LogType := ALWAYS ; -- Log if LogType is enabled
  Enable      : boolean := FALSE    -- also log if Enable is TRUE
) ;

-- without an AlertLogID
procedure log(
  Message  : string ;
  Level    : LogType := ALWAYS ; -- Log if LogType is enabled
  Enable   : boolean := FALSE    -- also log if Enable is TRUE
) ;
```

Usage:

```
Log(UartID, "Uart Parity Received", DEBUG) ;
.
.
Log("Received UART word", DEBUG) ;
```

9.3 LogHeader

LogHeader is a log message plus prefix and suffix lines to frame the message and make it stand out. Square brackets below indicate the parameter is optional.

```
procedure LogHeader (
  [ AlertLogID  : AlertLogIDType ; ]
  Message      : string ;
  Prefix       : string ;
  Suffix       : string ;
  Level        : LogType := ALWAYS ;
  Enable       : boolean := FALSE
) ;
```

Each character in the prefix and suffix indicates a line to print. Each line printed is prefixed by OSVVM_PRINT_PREFIX. If the character is a space, then print a blank line, otherwise, repeat that character OSVVM_LINE_LENGTH times.

There is also a short form that uses OSVVM_HEADER_PREFIX (default " ") and OSVVM_HEADER_SUFFIX (default "*") instead of having a Prefix and Suffix parameter.

```

procedure LogHeader (
  [ AlertLogID   : AlertLogIDType ; ]
  Message       : string ;
  Level         : LogType := ALWAYS ;
  Enable        : boolean := FALSE    -- override internal enable
) ;

```

Usage:

```
LogHeader(TB, "Start Of UART1 Test", " *", "* ") ;
```

Prints as (line length shortened to fit page width):

```

%%
%% *****
%%          0.0 ns  Log    ALWAYS  in TB,      Start of UART1 Test
%% *****
%%

```

10 Alert Method Reference

Alert is intended for parameter error checking. For self-checking see affirmations.

10.1 AlertType

Alert levels can be FAILURE, ERROR, or WARNING.

```
type AlertType is (FAILURE, ERROR, WARNING) ;
```

10.2 Alert

Alert generates an alert. The following overloading is supported.

```
procedure alert(  
  AlertLogID : AlertLogIDType ;  
  Message    : string ;  
  Level      : AlertType := ERROR  
) ;  
  
procedure Alert( Message : string ; Level : AlertType := ERROR ) ;
```

Usage of alert:

```
....  
Alert(UartID, "Uart Parity", ERROR) ;  
  
Alert("Uart Parity") ; -- ERROR by default
```

10.3 AlertIf and AlertIfNot

Alert has two conditional forms, AlertIf and AlertIfNot. The following is their overloading.

```
-- with an AlertLogID  
procedure AlertIf( AlertLogID : AlertLogIDType ; condition : boolean ;  
  Message : string ; Level : AlertType := ERROR ) ;  
impure function AlertIf( AlertLogID : AlertLogIDType ; condition : boolean ;  
  Message : string ; Level : AlertType := ERROR ) return boolean ;  
procedure AlertIfNot( AlertLogID : AlertLogIDType ; condition : boolean ;  
  Message : string ; Level : AlertType := ERROR ) ;  
impure function AlertIfNot( AlertLogID : AlertLogIDType ; condition : boolean ;  
  Message : string ; Level : AlertType := ERROR ) return boolean ;  
  
-- without an AlertLogID  
procedure AlertIf( condition : boolean ;  
  Message : string ; Level : AlertType := ERROR ) ;  
impure function AlertIf( condition : boolean ;  
  Message : string ; Level : AlertType := ERROR ) return boolean ;  
procedure AlertIfNot( condition : boolean ;  
  Message : string ; Level : AlertType := ERROR ) ;  
impure function AlertIfNot( condition : boolean ;  
  Message : string ; Level : AlertType := ERROR ) return boolean ;
```

Usage of conditional alerts:

```

-- with an AlertLogID
AlertIf(UartID, Break='1', "Uart Break", ERROR) ;
AlertIfNot(UartID, ReadValid, "Read", FAILURE);

-- without an AlertLogID
AlertIf(Break='1', "Uart Break", ERROR) ;
AlertIfNot(ReadValid, "Read Failed", FAILURE) ;

```

The function form is convenient for use for conditional exit of a loop.

```

exit AlertIfNot(UartID, ReadValid, "in ReadCovDb while reading ...", FAILURE) ;

```

10.4 AlertIfEqual and AlertIfNotEqual

AlertIfEqual and AlertIfNotEqual checks two values and produces an Alert if the check fails. In the following, AType can be std_logic, std_logic_vector, unsigned, signed, integer, integer_vector, real, character, string, or time.

```

-- with an AlertLogID
procedure AlertIfEqual( AlertLogID : AlertLogIDType ; L, R : AType ;
    Message : string ; Level : AlertType := ERROR ) ;
procedure AlertIfNotEqual( AlertLogID : AlertLogIDType ; L, R : AType ;
    Message : string ; Level : AlertType := ERROR ) ;

-- without an AlertLogID
procedure AlertIfEqual( L, R : AType ; Message : string ;
    Level : AlertType := ERROR ) ; Message : string ;
procedure AlertIfNotEqual( L, R : AType ;
    Level : AlertType := ERROR ) ;

```

10.5 AlertIfFilesNotMatch

Alert form AlertIfFilesNotMatch is for comparing two files.

```

-- with an AlertLogID
procedure AlertIfFilesNotMatch (AlertLogID : AlertLogIDType ;
    Name1, Name2 : string; Message : string := "" ; Level : AlertType := ERROR ;
    IgnoreSpaces : boolean := ALERT_LOG_IGNORE_SPACES ;
    IgnoreEmptyLines : boolean := ALERT_LOG_IGNORE_EMPTY_LINES ) ;
procedure AlertIfFilesNotMatch (AlertLogID : AlertLogIDType ;
    file File1, File2 : text; Message : string := "" ; Level : AlertType := ERROR ;
    IgnoreSpaces : boolean := ALERT_LOG_IGNORE_SPACES ;
    IgnoreEmptyLines : boolean := ALERT_LOG_IGNORE_EMPTY_LINES ) ;

-- without an AlertLogID
procedure AlertIfFilesNotMatch (
    Name1, Name2 : string; Message : string := "" ; Level : AlertType := ERROR ;
    IgnoreSpaces : boolean := ALERT_LOG_IGNORE_SPACES ;
    IgnoreEmptyLines : boolean := ALERT_LOG_IGNORE_EMPTY_LINES ) ;
procedure AlertIfFilesNotMatch (
    file File1, File2 : text; Message : string := "" ; Level : AlertType := ERROR ;
    IgnoreSpaces : boolean := ALERT_LOG_IGNORE_SPACES ;
    IgnoreEmptyLines : boolean := ALERT_LOG_IGNORE_EMPTY_LINES ) ;

```

10.6 IncrementAlertCount

Intended as a silent alert. Used by CoveragePkg.

```
-- Hierarchy
procedure IncAlertCount( -- A silent form of alert
    AlertLogID : AlertLogIDType ;
    Level      : AlertType := ERROR
) ;
-- Global Alert Counters
procedure IncAlertCount( Level : AlertType := ERROR ) ;
```

11 Affirmation Reference

Affirmations are a combination of Alerts and Logs. If the Affirmation is true, then a log is generated. If an Affirmation is false, then an alert is generated.

Affirmations are intended to be used for self-checking of a test. Each call to AffirmIf is counted and reported during ReportAlerts. This provides feedback on the amount of self-checking added by a test and is used as a quality metric.

11.1 AffirmIf / AffirmIfNot

AffirmIf has two forms of overloading. The first has a separate ReceivedMessage and ExpectedMessage. When the affirmation passes, log is called with just the ReceivedMessage. When an affirmation fails, alert is called with the ExpectedMessage concatenated to the end of the ReceivedMessage.

```
-- with an AlertLogID
procedure AffirmIf(
  AlertLogID      : AlertLogIDType ;
  condition       : boolean ;
  ReceivedMessage : string ;
  ExpectedMessage : string ;
  Enable          : boolean := FALSE
) ;

-- without an AlertLogID
procedure AffirmIf(
  condition       : boolean ;
  ReceivedMessage : string ;
  ExpectedMessage : string ;
  Enable          : boolean := FALSE
) ;
```

The second overloading of AffirmIf has a single Message parameter. Hence, both alert and log print the same message.

```
-- with an AlertLogID
procedure AffirmIf(
  AlertLogID      : AlertLogIDType ;
  condition       : boolean ;
  Message         : string ;
  Enable          : boolean := FALSE
) ;

-- without an AlertLogID
procedure AffirmIf(
  condition       : boolean ;
  Message         : string ;
  Enable          : boolean := FALSE
) ;
```

AffirmIfNot supports the same parameters as AffirmIf.

11.2 AffirmIfEqual and AffirmIfNotEqual

Affirmation form AffirmIfEqual checks if two values are equal. It greatly simplifies the message printing since it differentiates between the Received and Expected values. In the following, AType can be std_logic, std_logic_vector, unsigned, signed, integer, integer_vector, real, character, string or time.

```
-- with an AlertLogID
procedure AffirmIfEqual(
  AlertLogID      : AlertLogIDType ;
  Received, Expected : AType ;
  Message         : string := "";
  Enable          : boolean := FALSE
) ;

-- without an AlertLogID
procedure AffirmIfEqual(
  Received, Expected : AType ;
  Message         : string := "";
  Enable          : boolean := FALSE
) ;
```

AffirmIfNotEqual supports the same parameters as AffirmIfEqual.

AffirmIfEqual is effectively an abbreviation for,

```
AffirmIf (AlertLogId, Received = Expected,
  Message & " Received: " & to_string(Received),
  " /= Expected: " & to_string(Expected),
  Enable) ;
```

11.3 AffirmIfInRange

AffirmIfInRange checks if the received value is between the specified min and max values (inclusive). In the following, AType can be std_logic, std_logic_vector, unsigned, signed, integer, real, character, or time.

```
-- with an AlertLogID
procedure AffirmIfInRange (
  AlertLogID      : AlertLogIDType ;
  Received, Min, Max : AType ;
  Message         : string := "";
  Enable          : boolean := FALSE
) ;

-- without an AlertLogID
procedure AffirmIfInRange (
  Received, Min, Max : AType ;
  Message         : string := "";
  Enable          : boolean := FALSE
) ;
```

AffirmIfInRange is an abbreviation for,

```
AffirmIf(AlertLogID, (Min <= Received) and (Received <= Max),
  AddSpaceIfNotEmpty(Message) &
  "Check " & to_string(Min) & " <= " &
```

```
"Received (" & to_string(Received) & ") <= " & to_string(Max),
Enable) ;
```

11.4 AffirmIfFilesMatch

Affirmation form AffirmIfFilesMatch is for comparing two files.

```
-- with an AlertLogID
procedure AffirmIfFilesMatch (
  AlertLogID      : AlertLogIDType ;
  Name1, Name2    : string;
  Message         : string := "" ;
  Enable          : boolean := FALSE ;
  IgnoreSpaces    : boolean := ALERT_LOG_IGNORE_SPACES ;
  IgnoreEmptyLines : boolean := ALERT_LOG_IGNORE_EMPTY_LINES ) ;
procedure AffirmIfFilesMatch (
  AlertLogID      : AlertLogIDType ;
  file File1, File2 : text;
  Message         : string := "" ;
  Enable          : boolean := FALSE ;
  IgnoreSpaces    : boolean := ALERT_LOG_IGNORE_SPACES ;
  IgnoreEmptyLines : boolean := ALERT_LOG_IGNORE_EMPTY_LINES ) ;
-- without an AlertLogID
procedure AffirmIfFilesMatch (
  Name1, Name2    : string;
  Message         : string := "" ;
  Enable          : boolean := FALSE ;
  IgnoreSpaces    : boolean := ALERT_LOG_IGNORE_SPACES ;
  IgnoreEmptyLines : boolean := ALERT_LOG_IGNORE_EMPTY_LINES ) ;
procedure AffirmIfFilesMatch (
  file File1, File2 : text;
  Message         : string := "" ;
  Enable          : boolean := FALSE ;
  IgnoreSpaces    : boolean := ALERT_LOG_IGNORE_SPACES ;
  IgnoreEmptyLines : boolean := ALERT_LOG_IGNORE_EMPTY_LINES ) ;
```

11.5 AffirmIfTranscriptsMatch

AffirmIfTranscriptsMatch compares the current transcript file (defined by GetTranscriptName) with the same named file in the directory specified by the PathToValidatedResults parameter.

```
procedure AffirmIfTranscriptsMatch (
  AlertLogID      : AlertLogIDType ;
  PathToValidatedResults : string;
  Message         : string := "" ;
  Enable          : boolean := FALSE ;
  IgnoreSpaces    : boolean := ALERT_LOG_IGNORE_SPACES ;
  IgnoreEmptyLines : boolean := ALERT_LOG_IGNORE_EMPTY_LINES ) ;

procedure AffirmIfTranscriptsMatch (
  PathToValidatedResults : string;
  Message         : string := "" ;
  Enable          : boolean := FALSE ;
  IgnoreSpaces    : boolean := ALERT_LOG_IGNORE_SPACES ;
  IgnoreEmptyLines : boolean := ALERT_LOG_IGNORE_EMPTY_LINES ) ;
```

11.6 AffirmIfStable, AffirmIfNotStable

AffirmIfStable checks if the LastChange is greater than or equal to the Stable requirement.

```
-- with an AlertLogID
procedure AffirmIfStable(
  AlertLogID      : AlertLogIDType ;
  LastChange, Stable : time ;
  Message         : string := "";
  Enable          : boolean := FALSE
) ;

-- without an AlertLogID
procedure AffirmIfEqual(
  LastChange, Stable : time ;
  Message         : string := "";
  Enable          : boolean := FALSE
) ;
```

AffirmIfStable is an abbreviation for,

```
AffirmIf(AlertLogID, LastChange >= Stable,
  Message & "Check Stable: LastChange: " & format(LastChange) &
  " > Stable Requirement: " & format(Stable),
  Enable) ;
```

AffirmIfNotStable has the same parameters and checks that LastChange is less than the Stable requirement.

11.7 GetAffirmCount

Returns the current affirmation check count.

```
impure function GetAffirmCount return natural ;
```

11.8 IncAffirmCount

Increments the affirmation check count. Intended to be used only in special situations, such as packages that have additional considerations when using affirmations.

```
procedure IncAffirmCount ;
```

12 HTML Test Reports

12.1 Required Steps for Report Generation

Reports are created as a partnership of VHDL code and OSVVM scripts.

The VHDL code creates the YAML files by doing:

- Set the test name by calling SetTestName (AlertLogPkg),
- Do Alerts and Affirmations during the test (AlertLogPkg),
- At the end of the test, call EndOfTestReports

Within OSVVM scripting:

- Run each test case with OSVVM script API command simulate or RunTest.
- Run the entire test suite with OSVVM script API command build.

Details of the reports are shown in the "OSVVM Test Writer's User Guide" in the Documentation repository.

12.2 EndOfTestReports (in ReportPkg)

EndOfTestReports creates YAML files that OSVVM scripts use to create the reports.

The call interface to EndOfTestReports is:

```
procedure EndOfTestReports (  
    ReportAll      : boolean      := FALSE ;  
    ExternalErrors : AlertCountType := (0,0,0) ;  
    Stop           : boolean      := FALSE ;  
    TimeOut        : boolean      := FALSE  
);
```

If ReportAll is true, the text based test report prints details during pass (otherwise it only prints details during an error).

Test pass/fail is determined by AlertErrors + ExternalErrors – ExpectedErrors. If the count of FAILURE, ERROR, or WARNING is non-zero the test case fails. AlertErrors are those tabulated by Alerts and Affirmations as the test is running. ExternalErrors is the parameter of EndOfReports – and may be negative also to indicated expected errors. ExpectedErrors is the value set by SetExpectedAlertCount (this section) or IncrementExpectedAlertCount.

If the Stop parameter is true (not recommended), std.env.stop is called.

If TimeOut is true, the test case reports TIMEOUT for status. This is treated as an error in the reports.

12.3 SetExpectedAlertCount

SetExpectedAlertCount sets the internal ExpectedErrors variable and is used when the test completes to calculate total errors. The call interface to SetExpectedAlertCount is:

```
procedure SetExpectedAlertCount (Count : AlertCountType) ;  
procedure SetExpectedAlertCount (Level : AlertType ; Count : integer) ;
```

These allow errors (FAILURE, ERROR, WARNING) to be specified either as a group or individually.

12.4 IncrementExpectedAlertCount

IncrementExpectedAlertCount increments the internal ExpectedErrors variable. The call interface to IncrementExpectedAlertCount is:

```
IncrementExpectedAlertCount (Level : AlertType ; Count : integer := 1) ;
```

12.5 GetExpectedAlertCount

GetExpectedAlertCount returns the value of the internal ExpectedErrors variable. The call interface to GetExpectedAlertCount is:

```
impure function GetExpectedAlertCount return AlertCountType ;
```

Note that if a single level is needed, the result value can be indexed.

```
ExpectedErrors := GetExpectedAlertCount(ERROR) ;
```

13 Requirements Tracking

OSVVM can track requirements either as an Alert or as Coverage. As an Alert, a requirement is nothing more than an AlertLogID that has a PASSED goal associated with it. As Coverage, a requirement is a CoverageID that has a goal. Requirements pass if the goal is reached. By default, the goal is 1.

Requirement IDs are created with NewReqID. A group of requirements can be read in with ReadSpecification. Then the ID can be looked up using GetReqID.

Requirements can be checked by using any affirmation and the AlertLogID that is associated with that requirement. Requirements can also be checked using a special form of AffirmIF that takes the requirement name (as a string).

When a test case completes and calls EndOfTestReports, a YAML requirements report is automatically generated. When a test suite completes, OSVVM scripts merge the YAML requirements for that test suite. When the build completes, OSVVM scripts merge the YAML requirements for each test suite that was run during the build and produces an HTML and CSV merged requirements report.

13.1 NewReqID

Use NewReqID when the intent is to create an AlertLogID that is used for requirements tracking. The interface for NewID is as follows.

```
impure function NewReqID(  
  Name          : string ;  
  Goal          : natural ;  
  ParentID      : AlertLogIDType      := REQUIREMENT_ALERTLOG_ID ;  
  ReportMode    : AlertLogReportModeType := ENABLED ;  
  PrintParent   : AlertLogPrintParentType := PRINT_NAME_AND_PARENT ;  
  CreateHierarchy : boolean           := TRUE  
) return AlertLogIDType ;
```

Call NewReqID with the name of the requirement and the PassedGoal.

```
signal Uart1ReqID : AlertLogIDType ;  
.  
.  
.  
Uart1ReqID := NewReqID("UART_REQ_1", 2);
```

13.2 GetReqID

GetReqID returns an AlertLogID for a requirement. If an AlertLogID already exists for the requirement, GetReqID will return its AlertLogID. Use GetReqID when the intent is to look up the ID – but if it does not exist yet then create it. Typically GetReqID is called in the testbench, well after time 0, to look up an ID.

The interface for GetReqID is as follows.

```
impure function GetReqID (  
  Name          : string ;  
  PassedGoal    : integer := -1 ; -- If -1, use DefaultPassedGoalVar  
  ParentID      : AlertLogIDType := ALERTLOG_ID_NOT_ASSIGNED;  
  CreateHierarchy : Boolean := TRUE  
) return AlertLogIDType ;
```

When the ParentID defaults to ALERTLOG_ID_NOT_ASSIGNED, internally GetReqID changes this to REQUIREMENT_ALERTLOG_ID.

Creating AlertLogIDs for requirements is as follows. Since PassedGoal is not set in the call, the DefaultPassedGoalVar is used (default is 1). Change with SetAlertLogOptions(DefaultPassedGoal => 2).

```
signal Uart1ReqID : AlertLogIDType ;  
signal Uart2ReqID : AlertLogIDType ;  
  
...  
InitializeProc: process  
Begin  
  Uart1ReqID := GetReqID("UART_REQ_1");  
  Uart2ReqID := GetReqID("UART_REQ_2", 2);
```

13.3 ReadSpecification

ReadSpecification reads files in the following format. In the following items enclosed in "[]" are optional and are ignored.

```
<requirement ID Name> [, Requirement Description] [, Requirement PASSED Goal]  
<requirement2 ID Name> [, Requirement2 Description] [, Requirement2 PASSED Goal]  
...  
<requirementN ID Name> [, RequirementN Description] [, RequirementN PASSED Goal]
```

The interface for ReadSpecification is as follows.

```
procedure ReadSpecification (FileName : string ; PassedGoal : integer := -1) ;
```

A call to ReadSpecification:

```
ReadSpecification("Uart_Specification.txt") ;
```

If a requirement is not set, by default it will be 1. The default can be changed using SetAlertLogOptions(DefaultPassedGoal => 2). Alternately all requirements can be changed to the PassedGoal specified in the call to ReadSpecification (here 2):

```
ReadSpecification("Uart_Specification.txt", 2) ;
```

13.4 SetPassedGoal

The PassedGoal of an AlertLogID can be replaced by calling SetPassedGoal.

```
procedure SetPassedGoal(AlertLogID : AlertLogIDType ; PassedGoal : integer ) ;  
.  
.  
SetPassedGoal(Req1ID, 2) ;
```

13.5 AffirmIf for Requirements

For requirements, an additional form of AffirmIf that uses the requirements ID name is provided. It has two forms of overloading. The first has a separate ReceivedMessage and ExpectedMessage. When the affirmation passes, log is called with just the Received Message. When an affirmation fails, alert is called with the ExpectedMessage concatenated to the end of the ReceivedMessage.

```
-- with an AlertLogID  
procedure AffirmIf(  
  RequirementsIDName : string ;  
  condition           : boolean ;  
  ReceivedMessage     : string ;  
  ExpectedMessage     : string ;  
  Enable              : boolean := FALSE  
) ;
```

The second overloading of AffirmIF has a single Message parameter. Hence, both alert and log print the same message.

```
-- with an AlertLogID  
procedure AffirmIf(  
  RequirementsIDName : string ;  
  condition           : boolean ;  
  Message            : string ;  
  Enable             : boolean := FALSE  
) ;
```

A call to AffirmIF with two parameters is as follows.

```
AffirmIF("T1.1", A > B, "Test 1.1 A (" & to_string(A) & ")",  
         "> B (" & to_string(B) & ")" ) ;
```

Note that the RequirementsIDName is case insensitive.

13.6 Using Other Forms of AffirmIF with Requirements

To use other forms of AffirmIF, either the ID needs to be known (NewReqID or GetReqID) or looked up using FindID.

```
AffirmIfEqual( Uart1ReqID, TestValue1, ExpectedValue1, "Message" ) ;  
AffirmIfEqual( FindID("Axi4Req1"), TestValue2, ExpectedValue2, "Message" ) ;
```


13.7 WriteRequirementsYaml (Called by EndOfTestReports)

For information only. WriteRequirementsYaml is called by EndOfTestReports. WriteRequirementsYaml creates a YAML report that contains all requirements information for a test case. The file, ./reports/<TestSuite>/<TestName>_req.yaml, holds the requirements results for a given test case.

The call interface to WriteRequirementsYaml is:

```
procedure WriteRequirementsYaml (  
    FileName      : string ;  
    AlertLogID    : AlertLogIDType := REQUIREMENT_ALERTLOG_ID ;  
    OpenKind      : File_Open_Kind := WRITE_MODE  
) ;
```

13.8 Test Failure Due to Missed Requirements

When a test has requirements, a test will fail if requirements are not met and FailOnRequirementErrors is true (default is TRUE). Disable using SetAlertLogOptions(FailOnRequirementErrors => TRUE).

A test failing due to FailOnRequirementErrors will result in a ReportAlerts such as the following.

```
%% 381100.0 ns DONE FAILED req_test1 Total Error(s) = 2 Failures: 0 Errors:  
0 Warnings: 0 Passed: 20 Affirmations Checked: 20 Requirements Passed: 2 of 4  
%% Default Failures: 0 Errors: 0 Warnings: 0 Passed: 0 of 0  
%% OSVVM Failures: 0 Errors: 0 Warnings: 0 Passed: 0 of 0  
%% Requirements Failures: 0 Errors: 0 Warnings: 0 Passed: 0 of 0  
%% T1.1 Failures: 0 Errors: 0 Warnings: 0 Passed: 1 of 1  
%% T1.2 Failures: 0 Errors: 0 Warnings: 0 Passed: 0 of 2  
%% T1.3 Failures: 0 Errors: 0 Warnings: 0 Passed: 2 of 3  
%% T1.4 Failures: 0 Errors: 0 Warnings: 0 Passed: 4 of 4
```

13.9 ReportAlerts and Requirements

ReportAlerts prints requirements in both the summary and the detailed report when a test fails.

In the summary report (first line) of ReportAlerts, a requirements summary (# met of # total) will be printed if either PrintRequirements is TRUE (default is FALSE) or both the test has requirements and PrintIfHaveRequirements is TRUE (default is TRUE).

```
%% 381100.0 ns DONE PASSED req_test2 Total Error(s) = 0 Failures: 0 Errors:  
0 Warnings: 0 Passed: 20 Affirmations Checked: 20 Requirements Passed: 4 of 4
```

For a test that fails, detailed printing of requirements will be printed in ReportAlerts if PrintRequirements is TRUE (default is FALSE).

```
%% 381100.0 ns DONE FAILED req_test1 Total Error(s) = 6 Failures: 0 Errors:  
4 Warnings: 0 Passed: 20 Affirmations Checked: 20 Requirements Passed: 2 of 4  
%% Default Failures: 0 Errors: 0 Warnings: 0 Passed: 0 of 0  
%% OSVVM Failures: 0 Errors: 0 Warnings: 0 Passed: 0 of 0  
%% Requirements Failures: 0 Errors: 4 Warnings: 0 Passed: 0 of 0
```

%%	T1.1	Failures: 0	Errors: 0	Warnings: 0	Passed: 1 of 1
%%	T1.2	Failures: 0	Errors: 2	Warnings: 0	Passed: 0 of 2
%%	T1.3	Failures: 0	Errors: 2	Warnings: 0	Passed: 1 of 3
%%	T1.4	Failures: 0	Errors: 0	Warnings: 0	Passed: 4 of 4

Both PrintRequirements and PrintIfHaveRequirements can be changed using SetAlertLogOptions.

14 Enabling and Disabling Logs and Alerts

14.1 SetLogEnable: Enable / Disable Logging

Excepting ALWAYS, log enables are disabled by default. SetLogEnable allows alert levels to be individually enabled. When used without AlertLogID, SetLogEnable sets a value for all AlertLogIDs.

```
procedure SetLogEnable(Level : LogType ; Enable : boolean) ;  
.  
.  
.  
SetLogEnable(PASSED, TRUE) ;
```

When an AlertLogID is used, SetLogEnable sets a value for that AlertLogID, and if Hierarchy is true, the AlertLogIDs of its children.

```
procedure SetLogEnable(AlertLogID : AlertLogIDType ;  
    Level : LogType ; Enable : boolean ; DescendHierarchy : boolean := TRUE) ;  
.  
.  
.  
--      AlertLogID, Level, Enable, DescendHierarchy  
SetLogEnable(UartID, INFO, TRUE, TRUE) ;
```

14.2 Reading Log Enables from a FILE

ReadLogEnables read enables from a file.

```
procedure ReadLogEnables (FileName : string) ;  
procedure ReadLogEnables (file AlertLogInitFile : text) ;
```

The preferred file format is:

```
U_CpuModel DEBUG  
U_UART_TX DEBUG INFO  
U_UART_RX PASSED INFO DEBUG
```

ReadLogEnables will also read a file of the format:

```
U_CpuModel  
DEBUG  
U_UART_TX  
DEBUG  
U_UART_TX  
INFO  
.  
.  
.
```

14.3 IsLogEnabled /GetLogEnable

IsLoggingEnabled returns true when logging is enabled for a particular AlertLogID.

```
impure function IsLogEnabled(Level : LogType) return boolean ;  
impure function IsLogEnabled(AlertLogID : AlertLogIDType ; Level : LogType)  
    return boolean ;  
.  
.  
.  
If IsLogEnabled(UartID, DEBUG) then  
.  
.  
.
```

GetLogEnable is a synonym for IsLogEnabled.

```
impure function GetLogEnable(AlertLogID : AlertLogIDType ; Level : LogType)
return boolean ;
impure function GetLogEnable(Level : LogType) return boolean ;
```

14.4 SetAlertEnable: Alert Enable / Disable

Alerts are enabled by default. SetAlertEnable allows alert levels to be individually enabled or disabled. When used without AlertLogID, SetAlertEnable sets a value for all AlertLogIDs.

```
procedure SetAlertEnable(Level : AlertType ; Enable : boolean) ;
. . .
-- Level, Enable
SetAlertEnable(WARNING, FALSE) ;
```

When an AlertLogID is used, SetAlertEnable sets a value for that AlertLogID, and if DescendHierarchy is TRUE, it's the AlertLogID's of its children.

```
procedure SetAlertEnable(AlertLogID : AlertLogIDType ; Level : AlertType ;
Enable : boolean ; DescendHierarchy : boolean := TRUE) ;
```

14.5 GetAlertEnable

Get the value of the current alert enable for either a specific AlertLogID or for the global alert counter.

```
-- Hierarchy
impure function GetAlertEnable(AlertLogID : AlertLogIDType ; Level : AlertType)
return boolean ;
-- Global Alert Counter
impure function GetAlertEnable(Level : AlertType) return boolean ;
```

14.6 ClearAlerts: Reset Alert Counts

ClearAlerts resets alert counts to 0. Recommended as an alternative to SetGlobalAlertEnable – just after the design enters reset, call ClearAlerts.

```
procedure ClearAlerts ;
```

14.7 ClearAlertCounts: Reset Alert and Alert Stop Counts

ClearAlerts resets alert counts to 0 and alert stop counts back to their default.

```
procedure ClearAlertCounts ;
```

14.8 SetGlobalAlertEnable: Alert Global Enable / Disable

SetGlobalAlertEnable allows Alerts to be globally enabled and disabled. The Global Alert Enable is enabled by default.

```
procedure SetGlobalAlertEnable (A : EnableType := TRUE) ;
impure function SetGlobalAlertEnable (A : EnableType := TRUE) return EnableType ;
```

Originally intended to be used to disable alerts during startup, as shown below. Instead, for most applications we recommend using ClearAlerts just after reset.

```
InitAlerts : Process
  constant DisableAlerts : boolean := SetGlobalAlertEnable(FALSE);
begin
  wait until nReset = '1' ; -- Deassertion of reset
  SetGlobalAlertEnable(TRUE) ; -- enable alerts
```

14.9 GetGlobalAlertEnable

Get the current value of the global alert enable

```
impure function GetGlobalAlertEnable return boolean ;
```

15 Stop Count and Print Count

15.1 SetAlertStopCount: Alert Stop Counts

SetAlertStopCount stops a simulation after a specified number of alerts have occurred. For tests that have numerous errors, this allows the test case to stop sooner and debugging to start.

If an alert stop count is reached, the simulation stops with status STOPLIMIT. When used without AlertLogID, SetAlertStopCount sets the alert stop count for the top level to the specified value.

```
procedure SetAlertStopCount(Level : AlertType ; Count : integer) ;  
.  
.  
.  
--           Level,      Count  
SetAlertStopCount(ERROR,      20) ; -- Stop if 20 errors occur
```

When used with an AlertLogID, SetAlertStopCount sets the value for the specified AlertLogID and adjusts the AlertStopCount at each parent by that amount.

```
procedure SetAlertStopCount(AlertLogID : AlertLogIDType ;  
    Level : AlertType ; Count : integer) ;  
.  
.  
.  
--           AlertLogID, Level,      Count  
SetAlertStopCount(UartID,      ERROR,      20) ;
```

By default, the AlertStopCount for WARNING and ERROR are integer'right, and FAILURE is 0.

15.2 GetAlertStopCount

Get the value of the current alert stop count for either a specific AlertLogId or for the global alert counter.

```
-- Hierarchy  
impure function GetAlertStopCount(  
    AlertLogID : AlertLogIDType ;  
    Level : AlertType  
) return integer ;  
-- Global Alert Stop Count  
impure function GetAlertStopCount(Level : AlertType) return integer ;
```

15.3 ClearAlertStopCounts: Reset Alert Stop Counts

ClearAlerts resets alert stop counts back to their default.

```
procedure ClearAlertStopCounts ;
```

15.4 SetAlertPrintCount

Sets a maximum number of times to print an Alert level for an AlertLogID. The AlertPrintCount is specified either as an integer value for a particular level or as an AlertCountType value that specifies a count for each Alert level. Applies only to that ID, hence, if a child ID does not exceed its count, it prints, independent of the AlertPrintCount of its parent. When an ID is not specified, the ID of the default bin is used (ALERTLOG_DEFAULT_ID).

```
procedure SetAlertPrintCount(  
    AlertLogID : AlertLogIDType ; Level : AlertType ; Count : integer) ;  
procedure SetAlertPrintCount(Level : AlertType ; Count : integer) ;  
procedure SetAlertPrintCount(  
    AlertLogID : AlertLogIDType ; Count : AlertCountType) ;  
procedure SetAlertPrintCount(Count : AlertCountType) ;
```

15.5 GetAlertPrintCount

Returns the AlertPrintCount for an ID – either as an integer value for a particular level or as an AlertCountType value that contains a count for each Alert level. When an ID is not specified, the ID of the default bin is used (ALERTLOG_DEFAULT_ID).

```
impure function GetAlertPrintCount(AlertLogID : AlertLogIDType ; Level :  
AlertType) return integer ;  
    impure function GetAlertPrintCount(Level : AlertType) return integer ;  
    impure function GetAlertPrintCount(AlertLogID : AlertLogIDType) return  
AlertCountType ;  
    impure function GetAlertPrintCountreturn AlertCountType ;
```

16 Getting Counts

16.1 AlertCountType

Alerts are stored as a value of AlertCountType.

```
subtype AlertIndexType is AlertType range FAILURE to WARNING ;
type    AlertCountType is array (AlertIndexType) of integer ;
```

CAUTION: When working with values of AlertCountType, be sure to use named association as the type ordering may change in the future.

```
variable ExternalErrors : AlertCountType ;
...
ExternalErrors := (FAILURE => 0, ERROR => 6, WARNING => 0) ;
```

16.2 GetAlertCount

GetAlertCount returns the AlertCount value at AlertLogID. GetAlertCount is overloaded to return either AlertCountType or integer.

```
impure function GetAlertCount(AlertLogID : AlertLogIDType := ALERTLOG_BASE_ID)
    return AlertCountType ;
impure function GetAlertCount(AlertLogID : AlertLogIDType := ALERTLOG_BASE_ID)
    return integer ;
...
TopTotalErrors := GetAlertCount ;           -- AlertCount for Top of hierarchy
UartTotalErrors := GetAlertCount(UartID) ;  -- AlertCount for UartID
```

16.3 GetEnabledAlertCount

GetEnabledAlertCount is similar to GetAlertCount except it returns 0 for disabled alert levels. GetEnabledAlertCount is overloaded to return either AlertCountType or integer.

```
impure function GetEnabledAlertCount(AlertLogID : AlertLogIDType :=
    ALERTLOG_BASE_ID) return AlertCountType ;
impure function GetEnabledAlertCount (AlertLogID : AlertLogIDType :=
    ALERTLOG_BASE_ID) return integer ;
...
TopTotalErrors := GetEnabledAlertCount ;      -- Top of hierarchy
UartTotalErrors := GetEnabledAlertCount(UartID) ;  -- UartID
```

16.4 GetDisabledAlertCount

GetDisabledAlertCount returns the count of disabled errors for either the entire design hierarchy or a particular AlertLogID. GetDisabledAlertCount is relevant since a "clean" passing design will not have any disabled alert counts.

```
impure function GetDisabledAlertCount return AlertCountType ;
impure function GetDisabledAlertCount return integer ;
impure function GetDisabledAlertCount(AlertLogID: AlertLogIDType)
    return AlertCountType ;
impure function GetDisabledAlertCount(AlertLogID: AlertLogIDType) return integer ;
```


Note that disabled errors are not added to higher levels in the hierarchy. Hence, often $\text{GetAlertCount} \neq \text{GetEnabledAlertCount} + \text{GetDisabledAlertCount}$.

16.5 Math on AlertCountType

```
function "+" (L, R : AlertCountType) return AlertCountType ;
function "-" (L, R : AlertCountType) return AlertCountType ;
function "-" (R : AlertCountType) return AlertCountType ;
. . .
TotalAlertCount := Phase1Count + Phase2Count ;
TotalErrors := GetAlertCount - ExpectedErrors ;
NegateErrors := -ExpectedErrors ;
```

16.6 SumAlertCount: AlertCountType to Integer Error Count

SumAlertCount sums up the FAILURE, ERROR, and WARNING values into a single integer value.

```
impure function SumAlertCount(AlertCount: AlertCountType) return integer ;
. . .
ErrorCountInt := SumAlertCount(AlertCount) ;
```

17 Alert and Log Prefix and Suffix to Message

A prefix and suffix can be added message that is printed with Alert or Log. These are set on an AlertLogID basis using the following operations.

```
procedure SetAlertLogPrefix(AlertLogID : AlertLogIDType; Name : string ) ;
procedure UnSetAlertLogPrefix(AlertLogID : AlertLogIDType) ;
impure function GetAlertLogPrefix(AlertLogID : AlertLogIDType) return string ;
procedure SetAlertLogSuffix(AlertLogID : AlertLogIDType; Name : string ) ;
procedure UnSetAlertLogSuffix(AlertLogID : AlertLogIDType) ;
impure function GetAlertLogSuffix(AlertLogID : AlertLogIDType) return string ;
```

18 Text Based Test Case Reports via ReportAlerts

For most purposes calling EndOfTestReports and using the default text based reports is sufficient – particularly if you are using OSVVM scripting which also produces an HTML test case report that provides a superset of the information in the text based test case reports.

On the other hand, if you are using your own scripts to run the test cases, you may wish to use additional options and/or settings with ReportAlerts.

18.1 Report Summary

The report summary is the first line printed by ReportAlerts/ReportNonZeroAlerts

```
%%      381100.0 ns  DONE  FAILED  t17_print_ctrl  Total Error(s) = 2  Failures: 0
Errors: 0  Warnings: 0  Total Disabled Error(s) = 2  Failures: 0  Errors: 2
Warnings: 0  Passed: 0  Affirmations Checked: 2
```

Note in the above message, two affirmations were done and they both resulted in an error, however, the error was disabled (see SetAlertEnable) so it reported as a Disabled Error.

Options can be enabled or disabled using SetAlertLogOptions. Let's look at the report piece by piece.

If Total Errors $\neq 0$, then the following part of the message prints. In this case, Total Errors is nonzero since FailedOnDisabledErrors is enabled (default) and the disabled errors count as test errors. Hence, Failures, Errors, and Warnings are all 0.

```
Total Error(s) = 2  Failures: 0  Errors: 0  Warnings: 0
```

If FailedOnDisabledErrors is disabled, then the same test will PASS as shown below.

```
%%      381100.0 ns  DONE  PASSED  t17_print_ctrl  Total Disabled Error(s) = 2
Passed: 0  Affirmations Checked: 2
```

Next is the Total Disabled Errors summary. It will print if the DisabledAlerts $\neq 0$ or if PrintDisabledAlerts is enabled (default is disabled).

```
Total Disabled Error(s) = 2
```

Next is the details of the Disabled Errors. These will print if FailedOnDisabledErrors is enabled and DisabledAlerts $\neq 0$ or if PrintDisabledAlerts is enabled.

```
Total Disabled Error(s) = 2  Failures: 0  Errors: 2  Warnings: 0
```

Next is the printing of the pass and affirmation counts. If PrintPassed is enabled (default), the passed count will be printed. If PrintAffirmations is enabled (default is disabled) or Affirmation Count $\neq 0$ then the passed and affirmation counts will be printed.

```
Passed: 0  Affirmations Checked: 2
```

18.2 Report Details

When in hierarchy mode, a summary for each AlertLogID will be printed. This can include the following details. Each line summary starts with "%%".

```
%%      Default   Failures: 0 Errors: 2 Warnings: 0 Disabled Failures: 0 Errors:
0 Warnings: 0 Passed: 2 Affirmations: 4
%%      OSVVM     Failures: 0 Errors: 0 Warnings: 0 Disabled Failures: 0 Errors:
0 Warnings: 0 Passed: 0 Affirmations: 0
%%      Cpu1      Failures: 0 Errors: 2 Warnings: 0 Disabled Failures: 0 Errors:
0 Warnings: 0 Passed: 2 Affirmations: 4
%%      UartTx    Failures: 0 Errors: 0 Warnings: 0 Disabled Failures: 0 Errors:
2 Warnings: 0 Passed: 2 Affirmations: 4
%%      UartRx    Failures: 0 Errors: 2 Warnings: 0 Disabled Failures: 0 Errors:
0 Warnings: 0 Passed: 2 Affirmations: 4
```

The Failures, Errors, and Warnings always prints.

Disabled Failures, Errors, and Warnings only print if PrintDisabledAlerts is enabled (default is disabled).

Passed only prints if PrintPassed is enabled (default).

Affirmations only prints if PrintAffirmations is enabled (default is disabled).

With PrintDisabledAlerts disabled (default), PrintPassed enabled (default), and PrintAffirmations disabled (default), and DisabledAlerts = 0, the report will be simplified to the following.

```
%%      Default   Failures: 0 Errors: 2 Warnings: 0 Passed: 2
%%      OSVVM     Failures: 0 Errors: 0 Warnings: 0 Passed: 0
%%      Cpu1      Failures: 0 Errors: 2 Warnings: 0 Passed: 2
%%      UartTx    Failures: 0 Errors: 0 Warnings: 0 Passed: 2
%%      UartRx    Failures: 0 Errors: 2 Warnings: 0 Passed: 2
```

18.3 ReportAlerts: Reporting Alerts

At test completion call ReportAlerts to generate a simple text based report. If you call EndOfTestReports, it calls ReportAlerts, so you don't have to.

```
procedure ReportAlerts (
    Name           : string := "" ;
    AlertLogID      : AlertLogIDType := ALERTLOG_BASE_ID ;
    ExternalErrors  : AlertCountType := (others => 0) ;
    ReportAll       : Boolean         := FALSE
) ;
```

ReportAlerts has 4 optional parameters: Name, AlertLogID, ExternalErrors, and ReportAll. Name overrides the name specified by SetTestName. AlertLogID allows reporting alerts for a specific AlertLogID and its children (if any). ExternalErrors allows separately detected errors to be reported. ExternalErrors is type AlertCountType and the value (FAILURE => 0, ERROR => 5, WARNING => 1) indicates detection logic separate from AlertLogPkg saw 0 Failures, 5 Errors, and 1 Warning. See notes under

AlertCountType. When ReportAll is true, ReportAlerts reports a summary as well as details for each source – whether the test failed or passed.

```
--          Name,    AlertLogID,  ExternalErrors
ReportAlerts("Uart1", UartID,      (FAILURE => 0, ERROR => 5, WARNING => 1) ) ;
```

18.4 ReportNonZeroAlerts

When in hierarchy mode, a summary for each AlertLogID will be printed. This can include the following details. Each summary starts with "%%".

Within the hierarchy, if a level has no alerts set, then that level will not be printed.

```
procedure ReportNonZeroAlerts (
    Name          : string := OSVVM_STRING_INIT_PARM_DETECT ;
    AlertLogID     : AlertLogIDType := ALERTLOG_BASE_ID ;
    ExternalErrors : AlertCountType := (others => 0)
) ;
```

18.5 Overloaded ReportAlerts for Reporting AlertCounts

ReportAlerts can also be used to print a passed/failed message for an AlertCount that is passed into the procedure call. This will not use any of the internal settings or information.

```
procedure ReportAlerts ( Name : String ; AlertCount : AlertCountType) ;
```

This is useful to accumulate values returned by different phases of a test that need to be reported separately.

```
ReportAlerts("Test1: Final", Phase1AlertCount + Phase2AlertCount) ;
```

19 Alert and Log Output Control Options

All OSVVM Alert and Log output controls get their initial value from a settings in OsvvmSettingsPkg_default.vhd.

For most projects setting them in the OsvvmSettingsPkg for the entire project should be sufficient. These values can be customized for a project as described in the OSVVM Settings User Guide. This is the recommended approach.

19.1 Settings In OsvvmSettingsPkg_default.vhd

See OSVVM Settings User Guide for details on these and how to change them. This is the recommended approach to change these values. Note that there are additional values that do not have a setting.

19.2 SetAlertLogJustify

SetAlertLogJustify sets the justify variable. When the justify variable is true, the message of Alerts and Logs are justified so they align. Default value is determined by the setting ALERT_LOG_JUSTIFY_ENABLE.

```
SetAlertLogJustify(Enable : boolean := TRUE) ;
```

19.3 OsvvmOptionsType

OsvvmOptionsType defines the values for options. User values are: OPT_DEFAULT, DISABLED, FALSE, ENABLED, TRUE. The values DISABLED and FALSE are handled the same. The values ENABLED and TRUE are treated the same. The value OPT_USE_DEFAULT causes the variable to use its default value. OsvvmOptionsType is defined in OsvvmGlobalPkg.

```
type OsvvmOptionsType is (OPT_INIT_PARM_DETECT, OPT_USE_DEFAULT, DISABLED, FALSE, ENABLED, TRUE) ;
```

19.4 Deprecated: SetAlertLogOptions

SetAlertLogOptions allows individual settings to be made at run time. For most of these this is not recommended. This may be removed entirely in the future or selected parameters removed.

The call interface for SetAlertLogOptions is shown below. Note that parameters that have already been deprecated and have no impact are not shown.

```
procedure SetAlertLogOptions (  
  FailOnWarning          : OsvvmOptionsType := OPT_INIT_PARM_DETECT;  
  FailOnDisabledErrors   : OsvvmOptionsType := OPT_INIT_PARM_DETECT;  
  FailOnRequirementErrors : OsvvmOptionsType := OPT_INIT_PARM_DETECT;  
  ReportHierarchy        : OsvvmOptionsType := OPT_INIT_PARM_DETECT;  
  WriteAlertErrorCount    : AlertLogOptionsType := OPT_INIT_PARM_DETECT;  
  WriteAlertLevel         : OsvvmOptionsType := OPT_INIT_PARM_DETECT;
```

```

WriteAlertName      : OsvvmOptionsType := OPT_INIT_PARM_DETECT;
WriteAlertTime      : OsvvmOptionsType := OPT_INIT_PARM_DETECT;
WriteLogErrorCount  : AlertLogOptionsType := OPT_INIT_PARM_DETECT;
WriteLogLevel       : OsvvmOptionsType := OPT_INIT_PARM_DETECT;
WriteLogName        : OsvvmOptionsType := OPT_INIT_PARM_DETECT;
WriteLogTime        : OsvvmOptionsType := OPT_INIT_PARM_DETECT;
PrintPassed         : AlertLogOptionsType := OPT_INIT_PARM_DETECT;
PrintAffirmations   : AlertLogOptionsType := OPT_INIT_PARM_DETECT;
PrintDisabledAlerts : AlertLogOptionsType := OPT_INIT_PARM_DETECT;
PrintRequirements   : AlertLogOptionsType := OPT_INIT_PARM_DETECT;
PrintIfHaveRequirements : AlertLogOptionsType := OPT_INIT_PARM_DETECT;
DefaultPassedGoal   : integer          := integer'left ;
WriteTimeLast       : OsvvmOptionsType := OPT_INIT_PARM_DETECT ;
TimeJustifyAmount   : integer          := integer'left
) ;

```

SetAlertOptions will change as AlertLogPkg evolves. Use of named association is required to ensure future compatibility. Only specify the values that need to change. Values not specified in the call will not change (hence the peculiar initial values on the parameters).

```

SetAlertLogOptions (
    FailOnWarning      => FALSE,
    FailOnDisabledErrors => FALSE
) ;

```

The following tables explain the options, their initial value and the corresponding constant in OsvvmSettingsPkg.

Options that impact what causes a test to fail:

Parameter / Constant Name	Description	Value
FailOnWarning ALERT_LOG_FAIL_ON_WARNING	Count warnings as test errors.	TRUE
FailOnDisabledErrors ALERT_LOG_FAIL_ON_DISABLED_ERRORS	If FailOnDisabledErrors is enabled and DisabledAlerts /= 0, then 1) Test Fails 2) DisabledAlerts are printed in the report summary. 3) DisabledAlerts print for each AlertLogID	TRUE
FailOnRequirementErrors ALERT_LOG_FAIL_ON_REQUIREMENT_ERRORS	If FailOnRequirementErrors is enabled, a Test fails if it does not reach its requirements (PASSED goals).	TRUE

Options that impact requirements:

Parameter / Constant Name	Description	Value
DefaultPassedGoal ALERT_LOG_DEFAULT_PASSED_GOAL	Set the Default Passed Goal for ReadSpecification and GetReqID	1

Options that impact information printed by both Alert and Log:

Parameter / Constant Name	Description	Value
WriteTimeLast ALERT_LOG_WRITE_TIME_FIRST	Write time last when TRUE or first when FALSE.	FALSE
TimeJustifyAmount ALERT_LOG_TIME_JUSTIFY_AMOUNT	Print level.	9

Options that impact information printed by Alert:

Parameter / Constant Name	Description	Value
WriteAlertErrorCount ALERT_LOG_WRITE_ERRORCOUNT	Print Error Count immediately after %% Alert	FALSE
WriteAlertLevel ALERT_LOG_WRITE_LEVEL	Print level.	TRUE
WriteAlertName ALERT_LOG_WRITE_NAME	Print AlertLogID name.	TRUE
WriteAlertTime ALERT_LOG_WRITE_TIME	Alerts print time.	TRUE

Options that impact information printed by Log:

Parameter / Constant Name	Description	Value
WriteLogErrorCount ALERT_LOG_WRITE_ERRORCOUNT	Print Error Count immediately after %% Log	FALSE
WriteLogLevel ALERT_LOG_WRITE_LEVEL	Print level.	TRUE
WriteLogName ALERT_LOG_WRITE_NAME	Print AlertLogID name.	TRUE
WriteLogTime ALERT_LOG_WRITE_TIME	Logs print time.	TRUE

Options that impact the text report printed by ReportAlerts:

Parameter / Constant Name	Description	Value
ReportHierarchy ALERT_LOG_REPORT_HIERARCHY	When multiple AlertLogIDs exist, print an error summary for each level.	TRUE
PrintPassed ALERT_LOG_PRINT_PASSED	If PrintPassed is enabled, print number of passed logs in the report summary and for each AlertLogID	TRUE
PrintAffirmations ALERT_LOG_PRINT_AFFIRMATIONS	If PrintAffirmations is enabled, print the number of affirmations in the report summary and for each AlertLogID	FALSE
PrintDisabledAlerts ALERT_LOG_PRINT_DISABLED_ALERTS	If PrintDisabledAlerts is Enabled, print DisabledAlerts in the report summary and for each AlertLogID	FALSE

PrintRequirements ALERT_LOG_PRINT_REQUIREMENTS	If PrintRequirements is enabled, then 1) In the summary of ReportAlerts, print a requirements summary (# met of # total). 2) In the detailed report, print a requirements ReportAlerts, print a requirements summary (# met of # total).	FALSE
PrintIfHaveRequirements ALERT_LOG_PRINT_IF_HAVE_REQUIREMENTS	If PrintIfHaveRequirements is enabled and the test has requirements, then in the summary of ReportAlerts, print a requirements summary (# met of # total).	TRUE

In the event you missed the earlier notice, SetAlertLogOptions is deprecated. It is recommended to use OsvvmSettingsPkg instead. See Osvvm Settings User Guide.

19.5 Deprecated: ReportAlertLogOptions: Print Report Options

Prints out AlertLogPkg Report Options.

19.6 Deprecated: Getting AlertLog Report Options

Report options can be retrieved with one of the Get functions below.

```
function GetAlertLogFailOnWarning      return AlertLogOptionsType ;
function GetAlertLogFailOnDisabledErrors return AlertLogOptionsType ;
function GetAlertLogReportHierarchy    return AlertLogOptionsType ;
function GetAlertLogHierarchyInUse     return AlertLogOptionsType ;
function GetAlertLogWriteAlertLevel    return AlertLogOptionsType ;
function GetAlertLogWriteAlertName     return AlertLogOptionsType ;
function GetAlertLogWriteAlertTime     return AlertLogOptionsType ;
function GetAlertLogWriteLogLevel      return AlertLogOptionsType ;
function GetAlertLogWriteLogName       return AlertLogOptionsType ;
function GetAlertLogWriteLogTime       return AlertLogOptionsType ;
```

Note while the following are still supported, they just return the constant from OsvvmSettingsPkg_default.vhd identified in SetAlertLogOptions

```
function GetAlertLogAlertPrefix      return string ;
function GetAlertLogLogPrefix        return string ;
function GetAlertLogReportPrefix     return string ;
function GetAlertLogDoneName         return string ;
function GetAlertLogPassName         return string ;
function GetAlertLogFailName         return string ;
```

20 Deallocating and Re-initializing the Data structure

20.1 DeallocateAlertLogStruct

DeallocateAlertLogStruct deallocates all temporary storage allocated by AlertLogPkg. Also see ClearAlerts.

20.2 InitializeAlertLogStruct

InitializeAlertLogStruct is used after DeallocateAlertLogStruct to create and initialize internal storage.

21 Deprecated: Older CSV Type Reporting

OSVVM can also create Test Completion and Requirements reports as CSV files. The approach using CSV files requires considerable effort on your part.

These have been deprecated in favor of the VHDL (YAML) + Scripting (HTML, JUnit, CSV) approach that happen mostly autonomously with OSVVM VHDL + OSVVM Scripting.

21.1 Deprecated Reporting Test Completion: CSV Reports

As an alternative to creating YAML reports (from VHDL) and HTML and Junit reports (from OSVVM Scripting), OSVVM has an older CSV based approach.

This approach requires a significant amount of work that is specific to your test suite. As a result, it is deprecated.

The process for CSV based test tracking involves the following steps:

- WriteTestSummary – write out a one-line test summary. Each test in a test suite writes to the same results file.
- ReadTestSummaries – read in a set of test summaries written out by WriteTestSummary.
- ReportTestSummaries – Print a summary for each test that was read by ReadTestSummaries

In addition, WriteAlerts produces a CSV based detailed alert report.

CSV was explored as an initial methodology to write out reports for consumption by other tools. Unfortunately, CSV does not do an adequate job in conveying hierarchical relationships and it has been set aside in preference of YAML reports.

21.1.1 Deprecated: WriteAlerts

WriteAlerts prints all the information in ReportAlerts in a CSV format.

```
procedure WriteAlerts (  
    FileName      : string ;  
    AlertLogID    : AlertLogIDType := ALERTLOG_BASE_ID ;  
    OpenKind      : File_Open_Kind := WRITE_MODE  
) ;
```

The first line of WriteAlerts prints the report summary in a CSV format (see WriteTestSummary). The format of WriteAlerts is as follows:

```
<Name>,<PassedGoal>,<PassedCount>,<TotalErrors>,<Failures>,<Errors>,<Warnings>,<AffirmCount>
```

21.1.2 Deprecated: WriteTestSummary

WriteTestSummary creates a one line print of the information that is in the report summary (the first line of ReportAlerts) as a CSV. WriteTestSummary is intended to be used by each separate test in the test suite.

```
procedure WriteTestSummary(  
  FileName : string ;  
  OpenKind : File_Open_Kind := APPEND_MODE ) ;
```

The following WriteTestSummary adds a test summary for the current test to the OsvvmTestSummary.txt file. The default "APPEND_MODE" is used by all tests except maybe the first test. If the regression manager does not delete the OsvvmTestSummary.txt, then the first test will use "WRITE_MODE".

```
WriteTestSummary("./results/OsvvmTestSummary.txt") ;
```

WriteTestSummary prints in the following format. ReqGoal is the count of the number of requirements that have PassedGoals. ReqCount is the count of the number of requirements whose PassedCount is greater or equal to the PassedGoal.

```
<Name>, <ReqGoal>, <ReqCount>, <TotalErrors>, <Failures>, <Errors>, <Warnings>, <AffirmCo  
unt>, <PassedCount>
```

21.1.3 Deprecated: ReadTestSummaries

ReadTestSummaries reads in the file created by WriteTestSummary and records each separate test results as a requirement.

```
procedure ReadTestSummaries (FileName : string) ;
```

Calling ReadTestSummaries:

```
ReadTestSummaries("./results/OsvvmTestSummary.txt") ;
```

21.1.4 Deprecated: ReportTestSummaries

ReportTestSummaries produces a report of the results from the test summary file. ReportTestSummaries has no input parameters and is called as follows.

```
ReportTestSummaries ;
```

ReportTestSummaries produces a report such as the following. Note each line starts with %%. The "Affirmations ..." printed on a separate line is a line wrap in the text below.

```
%% PASSED test_1    Total Error(s) = 0  Failures: 0  Errors: 0  Warnings: 0  
Affirmations Passed: 11 of 11  Requirements Passed: 2 of 2  
%% FAILED test_2    Total Error(s) = 2  Failures: 0  Errors: 0  Warnings: 0  
Affirmations Passed: 22 of 22  Requirements Passed: 2 of 4  
%% PASSED test_3    Total Error(s) = 0  Failures: 0  Errors: 0  Warnings: 0  
Affirmations Passed: 33 of 33  Requirements Passed: 6 of 6  
%% FAILED test_4    Total Error(s) = 2  Failures: 0  Errors: 2  Warnings: 0  
Affirmations Passed: 44 of 44  Requirements Passed: 6 of 8  
%% PASSED test_5    Total Error(s) = 0  Failures: 0  Errors: 0  Warnings: 0  
Affirmations Passed: 55 of 55  Requirements Passed: 10 of 10
```

```
%% PASSED test_6    Total Error(s) = 0  Failures: 0  Errors: 0  Warnings: 0
Affirmations Passed: 66 of 66  Requirements Passed: 10 of 12
```

If a test does not run, it will fail if (AffirmCount <= 0) and (RequirementsGoal > RequirementsPassed) and (FailOnRequirementsVar = TRUE). FailOnRequirements is set by SetAlertLogOptions and TRUE is the default. These will print as follows:

```
%% FAILED test_7    Total Error(s) = 1  Failures: 0  Errors: 0  Warnings: 0
Affirmations Passed: 0 of 0  Requirements Passed: 0 of 1
%% FAILED test_8    Total Error(s) = 1  Failures: 0  Errors: 0  Warnings: 0
Affirmations Passed: 0 of 0  Requirements Passed: 0 of 1
```

If (AffirmCount = 0) and either (RequirementsGoal = 0) or FailOnRequirements is FALSE, the test results are indeterminant. These will print as follows:

```
%% ????? test_7    Total Error(s) = 1  Failures: 0  Errors: 0  Warnings: 0
Affirmations Passed: 0 of 0  Requirements Passed: 0 of 1
%% ????? test_8    Total Error(s) = 1  Failures: 0  Errors: 0  Warnings: 0
Affirmations Passed: 0 of 0  Requirements Passed: 0 of 1
```

21.1.5 Deprecated: WriteTestSummaries

WriteTestSummaries produces a csv file of the same information that is printed ReportTestSummaries.

```
procedure WriteTestSummaries ( FileName : string ; OpenKind : File_Open_Kind ) ;
```

21.1.6 Deprecated: Use Model

One potential use model is to just get a report of what passed and failed. This would involve the following.

```
-- In Test1.vhd
WriteTestSummary("./results/OsvvmTestSummary.txt", WRITE_MODE) ;
. . .
-- In Test2.vhd
WriteTestSummary("./results/OsvvmTestSummary.txt") ;
. . .
-- In TestN.vhd
WriteTestSummary("./results/OsvvmTestSummary.txt") ;

. . .
-- In OsvvmTestSummary.vhd
ReadTestSummaries ("./results/OsvvmTestSummary.txt") ;
ReportTestSummaries ;
-- WriteTestSummaries("PointLess_IsSameAs_OsvvmTestSummary.txt") ;
```

However, the above approach fails to find tests that for some reason did not run and did not produce any output. Note also that using, WriteTestSummaries in the above example would produce the same information that is already in OsvvmTestSummary.txt after all of the tests have finished. So there is no reason to do WriteTestSummaries above.

First, remember that test summaries are recorded as requirements. To find what tests did not run and to make the order of printing match a predefined order, let's define the tests that we wanted to run as a list of test names. Then read the name of tests in as a specification. Hence, we will modify `OsvvmTestSummary.vhd` as follows:

```
-- In OsvvmTestSummary.vhd
ReadSpecification ("./OsvvmTestsToRun.txt") ;
ReadTestSummaries ("./results/OsvvmTestSummary.txt") ;
ReportTestSummaries ;
WriteTestSummaries("OsvvmAllTestResults.txt") ;
```

In this case, tests that did not run will show up as having an `AffirmCount` of 0 and a `RequirementGoal` of 1, hence, by defaults of `ReportTestSummaries`, the test will fail.

21.2 Deprecated: Requirements Reporting: CSV Reports

`ReportRequirements` produces a test summary as well as a tabulation of each requirement. `WriteRequirements` prints the same information in a CSV format. `ReadRequirements` reads multiple files produced by `WriteRequirements` and merges them.

Caution: `ReportRequirements` and `WriteRequirements` only report/write requirements that are in the hierarchy under the `REQUIREMENT_ALERTLOG_ID`. The current implementation allows requirements to be anywhere in the `AlertLog` hierarchy.

21.2.1 Deprecated: ReportRequirements

`ReportRequirements` produces a report that has the test summary of the `ReportAlerts` and then just the IDs that are under the Requirements bin (via `GetReqID` or `ReadSpecification`). A `ReportRequirements` for the previous `ReportAlerts` is as follows.

```
ReportRequirements ;
. . .
%% 381100.0 ns DONE FAILED req_test1 Total Error(s) = 6 Failures: 0 Errors:
4 Warnings: 0 Passed: 20 Affirmations Checked: 20 Requirements Passed: 2 of 4
%% T1.1 Failures: 0 Errors: 0 Warnings: 0 Passed: 1 of 1
%% T1.2 Failures: 0 Errors: 2 Warnings: 0 Passed: 0 of 2
%% T1.3 Failures: 0 Errors: 2 Warnings: 0 Passed: 1 of 3
%% T1.4 Failures: 0 Errors: 0 Warnings: 0 Passed: 4 of 4
```

21.2.2 Deprecated: WriteRequirements

`WriteRequirements` creates a CSV file that has the same information as `ReportRequirements`. The format is identical to `WriteAlerts`.

```
procedure WriteRequirements (
  FileName      : string ;
  AlertLogID    : AlertLogIDType := REQUIREMENT_ALERTLOG_ID ;
  OpenKind      : File_Open_Kind := WRITE_MODE
) ;
```

21.2.3 Deprecated: ReadRequirements

ReadRequirements reads a file that is in the format of WriteRequirements. The intent of ReadRequirements is to read and merge the WriteRequirements information from multiple separate tests.

```
procedure ReadRequirements (  
    FileName      : string ;  
    ThresholdPassed : boolean := FALSE  
) ;
```

If the ThresholdPassed parameter is true and if the read PassedCount exceeds the read PassedGoal, the PassedCount is replaced by the PassedGoal.

22 Deprecated Names and their Replacement name

From time to time, names must change to keep consistency across a large set of packages. Where possible OSVVM uses a soft deprecation methodology where the subprogram takes on the new name and the original name is supported via an alias. For example SetTestName was originally named SetTestName and FindID was originally named FindAlertLogID. As a result, you will find aliases in AlertLogPkg as follows:

```
alias SetAlertLogName is SetTestName [string] ;  
alias FindAlertLogID is FindID [string return AlertLogIDType] ;
```

23 About AlertLogPkg

AlertLogPkg is part of the OSVVM Utility library.

AlertLogPkg was created by Jim Lewis of SynthWorks. Please support our work by buying your VHDL Training from SynthWorks.

AlertLogPkg.vhd is a work in progress and will be updated from time to time. Caution, undocumented items are experimental and may be removed in a future version.

24 Compiling and Using OSVVM Utility Library

Compilation order for OSVVM Utility Library is in CHANGELOG.md and osvvm.pro. To run osvvm.pro see the Script User Guide.

OSVVM Utility library is released under the Apache open source license. It is free (both to download and use - there are no license fees). You can download it from osvvm.org or from our development area on GitHub (<https://github.com/OSVVM/OSVVM>).

If you add features to the package, please donate them back under the same license as candidates to be added to the standard version of the package. If you need features, be sure to contact us.

We also support the OSVVM user community and blogs through <http://www.osvvm.org>. Interested in sharing about your experience using OSVVM? Let us know, you can blog about it at [osvvm.org](http://www.osvvm.org).

25 About the Author - Jim Lewis

Jim Lewis, the founder of SynthWorks, has thirty plus years of design, teaching, and problem solving experience. In addition to working as a Principal Trainer for SynthWorks, Mr Lewis has done ASIC and FPGA design, custom model development, and consulting.

Mr. Lewis is the primary developer of the Open Source VHDL Verification Methodology (OSVVM.org) packages and a participant in the IEEE VHDL standards. Neither of these activities generate revenue.

Please support our volunteer efforts by buying your VHDL training from SynthWorks.

If you find bugs these packages or would like to request enhancements, you can reach me at jim@synthworks.com.